Question 1:

The solution provides a stack-based postfix calculator that evaluates mathematical expressions efficiently. It supports basic arithmetic operations, variable assignment/retrieval, and handles invalid expressions with robust error checking. Using a stack and a symbol table, it processes tokens in postfix notation to compute results and manages variable values effectively.

Question 2:

- **Stack:** Used to store intermediate values during expression evaluation. A stack is suitable because postfix notation inherently requires a last-in, first-out (LIFO) structure for evaluating operators and operands in sequence.

- **Map:** Stores variables and their assigned values. A map allows efficient retrieval and updating of variable values, which is essential for supporting variable-based calculations.

- **Stringstream:** Parses the input line into tokens. It is suitable for breaking down user input into manageable parts for processing.

Question 3:

The calculator processes each expression by breaking it into smaller parts (tokens). Numbers are pushed onto a stack, while operators process the top numbers on the stack. For example, the "+" operator adds the top two numbers. Variables (like A or B) are stored in a table. If a variable is assigned a value (e.g., A = 10), it remembers this value for future calculations. The calculator also checks for errors, like dividing by zero or using invalid input, and clears the stack when an error occurs.

Question 4:

- **processToken(token):**

```
Input: A single token from the expression.

If token is a variable (A-Z):

    If variable exists in the symbol table:

        Push its value onto the stack.

    Else:

        Push the variable onto the stack.
```

```
Else if token is an operator:
    If token is "=":
        Pop two values from the stack (value, variable).
        If variable is valid (A-Z):
            Assign the value to the variable in the symbol table.
        Else:
            Throw an error.
    Else:
        Perform the operation using the top two stack values.
        Push the result onto the stack.
Else:
    Convert the token to a number and push it onto the stack.
If any error occurs:
    Clear the stack and throw the error
```

- **processLine(line):**

```
Input: A line containing a postfix expression.
Clear the stack.
Split the line into tokens.
For each token:
    Call processToken(token)
```

- **printStack():**

```
If the stack is empty:
    Print an empty list.
Else:
    Print the stack contents in order, formatted as a list.
```

Question 5:

- **processToken:**

```
        void processToken(const string& token)
        {
            try
            {
                if (isVariable(token))
                {
                    // If variable exists, push its value; if not, push
the variable itself
                    auto it = symbolTable.find(token[0]);
                    if (it != symbolTable.end())
                        calculationStack.push(it->second);
                    else
                        calculationStack.push(token[0]);
                }
                else if (isOperator(token))
                {
                    if (token == "=")
                    {
                        if (calculationStack.size() < 2)
                            throw runtime_error("Invalid assignment
operation");
                        double value = calculationStack.top();
calculationStack.pop();
                        double var = calculationStack.top();
calculationStack.pop();
                        char varName = static_cast<char>(var);
                        if (varName >= 'A' && varName <= 'Z')
                            symbolTable[varName] = value;
                        else
                            throw runtime_error("Invalid variable name");
                    }
                    else
                    {
                        double result = performOperation(token);
                        calculationStack.push(result);
                    }
                }
                else
                {  // Must be a number
                    try
                    {
                        double value = stod(token);
                        calculationStack.push(value);
                    }
                    catch (const invalid_argument&)
                    {
                        throw runtime_error("Invalid token: " + token);
                    }
                }
            }
```

```
        catch (const exception& e)
        {
            while (!calculationStack.empty()) calculationStack.pop();
            throw;
        }
    }
```

- **processLine:**

```
    //Processes a line of input and evaluates the postfix expression
    void processLine(const string& line)
    {
        // Clear stack before processing new line
        while (!calculationStack.empty())
            calculationStack.pop();

        istringstream iss(line);
        string token;

        while (iss >> token)
            processToken(token);
    }
```

- **printStack:**

```
    //Prints the current state of the calculation stack



    //If the stack is empty, prints an empty bracket pair

    void printStack() const

    {

        if (calculationStack.empty())

        {

            cout << "> []" << endl;

            return;

        }

        stack<double> temp = calculationStack;

        cout << "> [";

        bool first = true;

        string output;

        while (!temp.empty())
```

```
        {
            if (!first) output = " " + output;
            output = to_string(temp.top()) + output;
            first = false;
            temp.pop();
        }
        // Remove trailing zeros and decimal point if not needed
        for (size_t i = 0; i < output.length(); i++)
        {
            if (output[i] == '.')
            {
                size_t j = output.length() - 1;
                while (j > i && output[j] == '0') j--;
                if (j == i) j--;
                output = output.substr(0, j + 1);
                break;
            }
        }
        cout << output << "]" << endl;
    }
```

Question 6:

>> Video

Question 7:

- **Floating-Point Precision Issues**:
  When printing results, the current implementation may display imprecise floating-point values due to the inherent limitations of the `double` type (e.g., `1.0000000001` instead

of `1.0`).
Remedy:
Use `std::fixed` and `std::setprecision` to control the output format, ensuring cleaner results.

- **Clearing the Stack on Errors**:
  Clearing the entire stack upon encountering an error may result in loss of intermediate computations.
  Remedy:
  Instead of clearing the stack, retain its state and let the user decide whether to reset it. Provide a `reset` command if needed.

- **No Support for Parentheses or Mixed Notation**:
  The calculator does not support infix notation or parentheses, which might limit its usability for users unfamiliar with postfix notation.
  Remedy:
  Provide an infix-to-postfix conversion feature to allow users to input expressions in a more familiar format.

- **No Undo/Redo Mechanism**:
  Users have no way to revert or redo operations if they make a mistake or wish to backtrack.
  Remedy:
  Implement an undo/redo mechanism by maintaining a history stack for previous states of the calculation stack.